

CSS GRID

Guia didático com os exemplos HTML e CSS fornecidos

Organização do conteúdo

- 1–3. Fundamentos: Grid Container, Grid Items e display: grid
- 4–16. Linhas, colunas, frações, repetição e espaçamento
- 17–23. Construção e posicionamento dos itens
- 24–25. Áreas nomeadas com grid-template-areas
- 26–34. Alinhamento de itens e trilhas
- 35–39. Dimensionamento e responsividade
- 40–41. Grid x Flexbox e construção de uma página
- 42–43. Referência rápida de propriedades e funções
- 44–46. Atividades práticas baseadas nos exemplos fornecidos
- 47. Resumo final

1. Introdução

O CSS Grid Layout, normalmente chamado apenas de CSS Grid, é um sistema de layout do CSS utilizado para organizar elementos em linhas e colunas. Com Grid podemos criar páginas, galerias, conjuntos de cards, dashboards, menus, formulários, áreas de conteúdo e layouts completos.

Uma das principais características do Grid é permitir o controle simultâneo de linhas e colunas. Podemos visualizar a grade como uma tabela formada por linhas e colunas, na qual cada elemento pode ocupar uma ou mais células.

Exemplo fornecido: uma grade com linhas e colunas

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

O exemplo real utiliza três colunas. A coluna central recebe duas frações do espaço, enquanto as laterais recebem uma fração cada.

HTML

```
<div class="grade colunas">
  <div class="item">1fr</div>
  <div class="item">2fr</div>
  <div class="item">1fr</div>
</div>
```

CSS

```
.colunas {
  grid-template-columns: 1fr 2fr 1fr;
}
```

2. Conceito fundamental: Grid Container e Grid Items

Para trabalhar com CSS Grid precisamos compreender dois conceitos. O Grid Container é o elemento que recebe display: grid. A partir daí, seus filhos diretos passam a ser Grid Items e podem ser organizados pela grade.

Identificando container e itens

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

A div com classe grade é o Grid Container. As três divs com classe item são Grid Items porque estão diretamente dentro do container.

HTML

```
<div class="grade colunas">
  <div class="item">1fr</div>
  <div class="item">2fr</div>
  <div class="item">1fr</div>
</div>
```

CSS

```
.grade {
  display: grid;
  gap: 12px;
}

.colunas {
  grid-template-columns: 1fr 2fr 1fr;
}
```

3. Ativando o CSS Grid

A propriedade principal é display: grid. Sem ela, as propriedades específicas do Grid não transformam o elemento em um Grid Container.

Aplicação no exemplo fornecido

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

No arquivo fornecido, .grade é o container e recebe display: grid. A classe .repeticao define como suas colunas serão distribuídas.

HTML

```
<div class="grade repeticao">
  <div class="item">A</div>
  <div class="item">B</div>
  <div class="item">C</div>
  <div class="item">D</div>
</div>
```

CSS

```
.grade {
  display: grid;
  gap: 12px;
  padding: 12px;
}

.repeticao {
  grid-template-columns: repeat(4, minmax(100px, 1fr));
}
```

4. Criando colunas com grid-template-columns

A propriedade grid-template-columns define a estrutura das colunas do Grid. Em vez do exemplo genérico com três larguras fixas, o arquivo fornecido demonstra diretamente a divisão proporcional do espaço.

Colunas com diferentes proporções

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

A grade possui três colunas. A segunda recebe duas frações, portanto ocupa o dobro do espaço de cada coluna lateral.

HTML

```
<div class="grade colunas">
  <div class="item">1fr</div>
  <div class="item">2fr</div>
  <div class="item">1fr</div>
</div>
```

CSS

```
.colunas {
  grid-template-columns: 1fr 2fr 1fr;
}
```

5. Criando colunas com porcentagem

O documento-base apresenta porcentagens como uma possibilidade de dimensionamento. Nos arquivos fornecidos, a mesma ideia de divisão do espaço aparece de forma mais flexível por meio da unidade fr. Assim, este material prioriza o exemplo real enviado em vez de criar um novo exemplo em porcentagem.

Exemplo equivalente fornecido com fr

Arquivo(s) utilizado(s): grid_basico.css

A unidade fr permite dividir o espaço disponível proporcionalmente, evitando a necessidade de definir porcentagens manualmente.

CSS

```
.colunas {
  grid-template-columns: 1fr 2fr 1fr;
}
```

6. A unidade fr

A unidade fr significa fraction, ou seja, fração do espaço disponível. Ela é especialmente útil quando queremos que as colunas sejam flexíveis.

Exemplo real com 1fr e 2fr

Arquivo(s) utilizado(s): tamanho_trilhas.html / tamanho_trilhas.css

O espaço disponível é dividido em quatro partes proporcionais: uma para a primeira coluna, duas para a segunda e uma para a terceira.

HTML

```
<div class="grade fracoes">
  <div class="item">1fr</div>
  <div class="item">2fr</div>
  <div class="item">1fr</div>
</div>
```

CSS

```
.fracoes {
  grid-template-columns: 1fr 2fr 1fr;
}
```

7. Diferentes proporções com fr

Valores diferentes de fr criam proporções diferentes entre as trilhas. No exemplo fornecido, 1fr 2fr 1fr produz uma coluna central duas vezes maior que cada coluna lateral.

Comparação visual das frações

Arquivo(s) utilizado(s): tamanho_trilhas.css

Podemos interpretar a configuração como quatro partes do espaço total: 1 + 2 + 1. A segunda coluna recebe duas dessas partes.

CSS

```
.fracoes {
  grid-template-columns: 1fr 2fr 1fr;
}
```

8. Exemplo prático com fr

O arquivo tamanho_trilhas.html mostra a unidade fr em uma situação real, com três cartões dentro de uma grade.

HTML + CSS do exemplo

Arquivo(s) utilizado(s): tamanho_trilhas.html / tamanho_trilhas.css

Além do dimensionamento, o exemplo mostra gap para separar as células.

HTML

```
<div class="grade fracoes">
  <div class="item">1fr</div>
  <div class="item">2fr</div>
  <div class="item">1fr</div>
</div>
```

CSS

```
.grade {
  display: grid;
  gap: 12px;
  padding: 12px;
}

.fracoes {
  grid-template-columns: 1fr 2fr 1fr;
}
```

9. Criando linhas com grid-template-rows

Assim como podemos definir colunas, podemos definir linhas. A propriedade grid-template-rows estabelece o dimensionamento das linhas explícitas.

Linhas definidas no exemplo fornecido

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

São criadas três linhas explícitas, com alturas de 70px, 100px e 130px. Como há duas colunas, cada linha recebe dois itens.

HTML

```
<div class="grade linhas">
  <div class="item">Linha 1</div>
  <div class="item">Linha 1</div>
  <div class="item">Linha 2</div>
  <div class="item">Linha 2</div>
  <div class="item">Linha 3</div>
  <div class="item">Linha 3</div>
</div>
```

CSS

```
.linhas {
  grid-template-columns: repeat(2, 1fr);
  grid-template-rows: 70px 100px 130px;
}
```

10. Usando auto

O valor auto permite que o tamanho de uma trilha seja determinado conforme o conteúdo e o espaço disponível. Nos arquivos fornecidos, auto aparece principalmente em combinações com minmax() e no dimensionamento automático das trilhas implícitas.

Exemplo fornecido com trilhas automáticas

Arquivo(s) utilizado(s): fluxo_automatico.html / fluxo_automatico.css

Neste caso, grid-auto-rows cria linhas implícitas com altura mínima de 65px e máximo auto, permitindo que o conteúdo determine a altura quando necessário.

HTML

```
<div class="grade em-linhas">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item destaque">Ocupa 2 colunas</div>
</div>
```

CSS

```
.em-linhas {
  grid-template-columns: repeat(3, minmax(0, 1fr));
  grid-auto-flow: row;
  grid-auto-rows: minmax(65px, auto);
}
```

11. Criando várias colunas com repeat()

Quando precisamos repetir o mesmo padrão de coluna ou linha, repeat() reduz a repetição do código. A sintaxe geral é repeat(quantidade, tamanho).

Exemplo real com repeat()

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

O código cria quatro colunas com o mesmo padrão de dimensionamento: no mínimo 100px e, quando possível, uma fração do espaço disponível.

HTML

```
<div class="grade repeticao">
  <div class="item">A</div>
  <div class="item">B</div>
  <div class="item">C</div>
  <div class="item">D</div>
</div>
```

CSS

```
.repeticao {
  grid-template-columns: repeat(4, minmax(100px, 1fr));
}
```

12. Exemplo com repeat()

O exemplo de repetição também demonstra que repeat() pode ser combinado com minmax(). Isso torna a definição da grade mais compacta e flexível.

Repetição com limite mínimo

Arquivo(s) utilizado(s): grid_basico.css

Leia a declaração como: repita quatro vezes uma trilha que pode ter no mínimo 100px e crescer até ocupar uma fração do espaço.

CSS

```
.repeticao {  
  grid-template-columns: repeat(4, minmax(100px, 1fr));  
}
```

13. gap

A propriedade gap define o espaço entre os elementos do Grid, tanto entre linhas quanto entre colunas quando é usado com um único valor.

Espaçamento usado nos arquivos fornecidos

Arquivo(s) utilizado(s): grid_basico.css / tamanho_trilhas.css / alinhamento_grid.css

Os exemplos fornecidos usam gap para separar visualmente os itens sem precisar aplicar margens individualmente aos filhos.

CSS

```
.grade {  
  display: grid;  
  gap: 12px;  
}
```

14. column-gap

column-gap controla apenas o espaço entre as colunas. O documento-base apresenta essa propriedade como uma forma de separar especificamente o eixo horizontal.

Relação com os exemplos fornecidos

Arquivo(s) utilizado(s): Exemplos fornecidos utilizam gap

Os arquivos enviados optam, na maioria dos casos, pelo shorthand gap. Se a intenção for separar somente colunas, column-gap pode ser usado no lugar de gap.

CSS

```
.grade {  
  display: grid;  
  gap: 12px;  
}
```

15. row-gap

row-gap controla apenas o espaço entre as linhas. Assim como column-gap, ele pode ser utilizado quando precisamos de controle independente em cada direção.

Relação com os exemplos fornecidos

Arquivo(s) utilizado(s): Exemplos fornecidos utilizam gap

Os exemplos fornecidos usam gap como solução geral. row-gap seria a alternativa quando somente o espaçamento vertical precisasse ser alterado.

CSS

```
.grade {  
  display: grid;  
  gap: 12px;  
}
```

16. Diferença entre gap, row-gap e column-gap

Podemos resumir: gap controla os dois eixos; row-gap controla o espaço entre linhas; column-gap controla o espaço entre colunas. Os arquivos reais usam principalmente gap para manter o código simples e uniforme.

Exemplo dos arquivos fornecidos

Arquivo(s) utilizado(s): fluxo_automatico.css

Quando há dois valores, gap também pode controlar linhas e colunas separadamente. O importante é identificar se o espaçamento desejado é geral ou específico.

CSS

```
.grade {  
  display: grid;  
  gap: 10px;  
}
```

17. Exemplo completo de Grid

O conjunto de arquivos grid_basico.html e grid_basico.css reúne, em uma mesma página, três situações: colunas proporcionais, linhas explícitas e repetição com largura mínima.

Trecho real que reúne as três situações

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

Este é o exemplo-base mais adequado para iniciar os testes, pois permite observar várias propriedades em uma única página.

HTML

```
<!DOCTYPE html>  
<html lang="pt-br">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Grid: linhas e colunas</title>  
  <link rel="stylesheet" href="../../css/grid/grid_basico.css">  
</head>  
<body>  
  <main>  
    <h1>Grid: linhas e colunas</h1>  
    <p>Use <code>display: grid</code> para organizar itens em linhas e colunas.</p>
```

```

<section class="exemplo">
  <h2>Colunas com medidas diferentes</h2>
  <div class="grade colunas">
    <div class="item">1fr</div>
    <div class="item">2fr</div>
    <div class="item">1fr</div>
  </div>
</section>
<section class="exemplo">
  <h2>Linhas e colunas definidas</h2>
  <div class="grade linhas">
    <div class="item">Linha 1</div>
    <div class="item">Linha 1</div>
    <div class="item">Linha 2</div>
    <div class="item">Linha 2</div>
    <div class="item">Linha 3</div>
    <div class="item">Linha 3</div>
  </div>
</section>
<section class="exemplo">
  <h2>Repetição e largura mínima</h2>
  <div class="grade repeticao">
    <div class="item">A</div>
    <div class="item">B</div>
    <div class="item">C</div>
    <div class="item">D</div>
  </div>
</section>
</main>
</body>

</html>

```

CSS

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 24px;
  background: #f4f7fb;
  color: #182230;
  font-family: Arial, sans-serif;
}

main {
  max-width: 900px;
  margin: 0 auto;
}

.exemplo {
  margin: 32px 0;
}

.grade {
  display: grid;
  gap: 12px;
  padding: 12px;
  background: #dce5f0;
  border-radius: 10px;
}

```

```

}

.colunas {
  grid-template-columns: 1fr 2fr 1fr;
}

.linhas {
  grid-template-columns: repeat(2, 1fr);
  grid-template-rows: 70px 100px 130px;
}

.repeticao {
  grid-template-columns: repeat(4, minmax(100px, 1fr));
}

.item {
  display: grid;
  min-height: 70px;
  place-items: center;
  padding: 12px;
  background: #2463a6;
  border-radius: 8px;
  color: white;
  font-weight: bold;
}

@media (max-width: 560px) {
  .repeticao {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

```

18. Posicionando elementos com grid-column

Podemos fazer um item ocupar mais de uma coluna utilizando grid-column. O valor span indica quantas trilhas o item deve ocupar a partir da posição em que o Grid o coloca.

Exemplo real

Arquivo(s) utilizado(s): posicionamento_itens.html / posicionamento_itens.css

O cabeçalho começa na linha de coluna 1 e termina na linha 4, ocupando três colunas. O principal ocupa duas colunas a partir de sua posição.

HTML

```

<div class="grade linhas">
  <div class="item cabecalho">grid-column: 1 / 4</div>
  <div class="item principal">grid-column: span 2</div>
</div>

```

CSS

```

.cabecalho {
  grid-column: 1 / 4;
}

.principal {
  grid-column: span 2;
}

```

19. grid-column-start e grid-column-end

Também podemos determinar exatamente onde um item começa e termina horizontalmente. O exemplo fornecido usa a forma abreviada grid-column, que corresponde ao controle por linhas inicial e final.

Forma abreviada presente no exemplo

Arquivo(s) utilizado(s): posicionamento_itens.css

Essa declaração equivale conceitualmente a definir grid-column-start: 1 e grid-column-end: 4.

CSS

```
.cabecalho {  
    grid-column: 1 / 4;  
}
```

20. grid-column: 1 / 3 e intervalos de linhas

A sintaxe grid-column: início / fim define um intervalo de linhas do Grid. A numeração se refere às linhas que delimitam as colunas, e não diretamente às colunas como unidades isoladas.

Aplicação no exemplo fornecido

Arquivo(s) utilizado(s): posicionamento_itens.css

O item ocupa o espaço entre as linhas 1 e 4. Portanto, nesse caso, são três colunas.

CSS

```
.cabecalho {  
    grid-column: 1 / 4;  
}
```

21. grid-row

A propriedade grid-row permite controlar a posição vertical de um elemento, definindo entre quais linhas ele será colocado.

Exemplo real

Arquivo(s) utilizado(s): posicionamento_itens.css

O item lateral ocupa o espaço entre as linhas 2 e 4, portanto atravessa duas linhas da grade.

CSS

```
.lateral {  
    grid-row: 2 / 4;  
}
```

22. grid-row: span

Também podemos usar span para indicar quantas linhas um item deve ocupar. O exemplo de fluxo automático utiliza esse recurso para criar itens maiores.

Exemplo real com span em linha e coluna

Arquivo(s) utilizado(s): fluxo_automatico.css

O item .grande ocupa duas colunas e duas linhas. O item .alto ocupa duas linhas.

CSS

```
.grande {
  grid-column: span 2;
  grid-row: span 2;
}

.alto {
  grid-row: span 2;
}
```

23. Exemplo prático de grid-column e grid-row

O arquivo posicionamento_itens.html foi construído especificamente para demonstrar linhas numeradas, span, grid-area e linhas nomeadas.

HTML completo do exemplo

Arquivo(s) utilizado(s): posicionamento_itens.html / posicionamento_itens.css

Observe como diferentes itens da mesma grade recebem regras de posicionamento diferentes.

HTML

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Grid: posicionamento dos itens</title>
  <link rel="stylesheet" href="../../css/grid/posicionamento_itens.css">
</head>

<body>
  <main>
    <h1>Posicionamento dos itens</h1>
    <p>As linhas do Grid permitem posicionar itens, ocupar intervalos e sobrepor áreas.</p>

    <section class="exemplo">
      <h2>Linhas numeradas e span</h2>
      <div class="grade linhas">
        <div class="item cabecalho">grid-column: 1 / 4</div>
        <div class="item lateral">grid-row: 2 / 4</div>
        <div class="item principal">grid-column: span 2</div>
        <div class="item detalhe">grid-area: 3 / 3 / 4 / 5</div>
        <div class="item rodape">Posicionado pelas linhas</div>
      </div>
    </section>

    <section class="exemplo">
      <h2>Linhas nomeadas</h2>
      <div class="grade nomeadas">
```

```

        <div class="item conteudo">Entre as linhas nomeadas</div>
        <div class="item nota">Coluna lateral</div>
    </div>
</section>
</main>
</body>

</html>

```

CSS

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 24px;
    background: #f5f3fa;
    color: #28233a;
    font-family: Arial, sans-serif;
}

main {
    max-width: 900px;
    margin: 0 auto;
}

.exemplo {
    margin: 32px 0;
}

.grade {
    display: grid;
    gap: 10px;
    padding: 12px;
    background: #e2deed;
    border-radius: 10px;
}

.linhas {
    grid-template-columns: repeat(4, minmax(0, 1fr));
    grid-template-rows: repeat(3, minmax(75px, auto));
}

.item {
    display: grid;
    min-height: 65px;
    place-items: center;
    padding: 12px;
    background: #6246a3;
    border-radius: 8px;
    color: white;
    font-weight: bold;
    text-align: center;
}

.cabecalho {
    grid-column: 1 / 4;
}

.lateral {

```

```
    grid-row: 2 / 4;
    background: #397c85;
}

.principal {
    grid-column: span 2;
    background: #a54c62;
}

.detalhe {
    grid-area: 3 / 3 / 4 / 5;
    background: #bd7134;
}

.rodape {
    grid-column: 1 / -1;
    background: #41536f;
}

.nomeadas {
    grid-template-columns: [inicio] minmax(0, 2fr) [conteudo-fim lateral-inicio] minmax(140px, 1fr) [fim];
}

.conteudo {
    grid-column: inicio / conteudo-fim;
}

.nota {
    grid-column: lateral-inicio / fim;
    background: #397c85;
}

@media (max-width: 560px) {
    .linhas {
        grid-template-columns: repeat(2, minmax(0, 1fr));
    }

    .cabecalho,
    .principal,
    .detalhe,
    .rodape {
        grid-column: 1 / -1;
    }

    .lateral {
        grid-row: auto;
    }
}
```

24. grid-template-areas

Uma das formas mais interessantes de organizar layouts é utilizar grid-template-areas. Ela permite atribuir nomes às regiões do Grid e representar visualmente o desenho da página dentro do CSS.

Exemplo real de áreas nomeadas

Arquivo(s) utilizado(s): areas_template.css

A própria sequência de textos dentro de grid-template-areas mostra a organização: cabeçalho em toda a largura, três áreas no meio e rodapé novamente ocupando toda a largura.

CSS

```
.pagina {
  display: grid;
  grid-template-columns: 220px minmax(0, 1fr) 200px;
  grid-template-areas:
    "cabeçalho cabeçalho cabeçalho"
    "navegacao conteudo lateral"
    "rodape rodape rodape";
  gap: 14px;
}
```

25. Exemplo completo com grid-template-areas

Depois de nomear as regiões, cada elemento recebe sua área com grid-area. Isso torna o relacionamento entre HTML e CSS fácil de ler.

HTML + CSS reais

Arquivo(s) utilizado(s): areas_template.html / areas_template.css

Além do layout para telas maiores, o mesmo exemplo reorganiza as áreas em 720px e 480px, mostrando como a estrutura pode mudar sem alterar o HTML.

HTML

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Grid: áreas nomeadas</title>
  <link rel="stylesheet" href="../../css/grid/areas_template.css">
</head>

<body>
  <main>
    <h1>Áreas nomeadas</h1>
    <p>Com <code>grid-template-areas</code>, o desenho do layout fica visível no CSS.</p>

    <div class="pagina">
      <header class="cabeçalho">Cabeçalho</header>
      <nav class="navegacao">Navegação</nav>
      <aside class="lateral">Conteúdo relacionado</aside>
      <article class="conteudo">
        <h2>Conteúdo principal</h2>
        <p>O mesmo HTML pode ser reorganizado alterando as áreas no CSS.</p>
      </article>
      <footer class="rodape">Rodapé</footer>
    </div>
```

```
</main>
</body>

</html>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 24px;
  background: #f4f6f8;
  color: #1c2733;
  font-family: Arial, sans-serif;
}

main {
  max-width: 1000px;
  margin: 0 auto;
}

.pagina {
  display: grid;
  grid-template-columns: 220px minmax(0, 1fr) 200px;
  grid-template-areas:
    "cabecalho cabecalho cabecalho"
    "navegacao conteudo lateral"
    "rodape rodape rodape";
  gap: 14px;
  margin-top: 24px;
}

.pagina > * {
  padding: 24px;
  border-radius: 10px;
}

.cabecalho {
  grid-area: cabecalho;
  background: #245b8f;
  color: white;
}

.navegacao {
  grid-area: navegacao;
  background: #d8e6f2;
}

.lateral {
  grid-area: lateral;
  background: #f3dfbf;
}

.conteudo {
  grid-area: conteudo;
  min-height: 240px;
  background: white;
}
```

```

.rodape {
  grid-area: rodape;
  background: #273b4f;
  color: white;
}

@media (max-width: 720px) {
  .pagina {
    grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
    grid-template-areas:
      "cabecalho cabecalho"
      "navegacao navegacao"
      "conteudo lateral"
      "rodape rodape";
  }
}

@media (max-width: 480px) {
  .pagina {
    grid-template-columns: 1fr;
    grid-template-areas:
      "cabecalho"
      "navegacao"
      "conteudo"
      "lateral"
      "rodape";
  }
}

```

26. justify-items

justify-items controla o alinhamento dos itens no eixo horizontal dentro de suas células. Os valores comuns incluem start, center, end e stretch.

Exemplo real de alinhamento

Arquivo(s) utilizado(s): alinhamento_grid.css

No exemplo, o comportamento padrão da grade é centralizar os itens horizontalmente dentro das respectivas células.

CSS

```

.itens {
  grid-template-columns: repeat(2, 1fr);
  grid-template-rows: repeat(2, 70px);
  justify-items: center;
}

```

27. align-items

align-items controla o alinhamento dos itens no eixo vertical dentro de suas células. Pode receber start, center, end e stretch.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.css

A declaração centraliza os itens verticalmente nas células da grade.

CSS

```
.itens {
  grid-template-columns: repeat(2, 1fr);
  grid-template-rows: repeat(2, 70px);
  align-items: center;
}
```

28. place-items

place-items é um atalho que combina justify-items e align-items. Quando usamos place-items: center, o item é centralizado horizontal e verticalmente.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.html / alinhamento_grid.css

O próprio conteúdo do HTML identifica o objetivo do exemplo: centralizar nos dois eixos.

HTML

```
<div class="grade place">
  <div class="item">Centro nos dois eixos</div>
</div>
```

CSS

```
.place {
  min-height: 180px;
  grid-template-columns: 1fr;
  grid-template-rows: 1fr;
  place-items: center;
}
```

29. justify-content

justify-content controla o alinhamento do Grid inteiro dentro do container no eixo horizontal. É diferente de justify-items: aqui o que se desloca é a grade/trilhas como conjunto.

Exemplo real com space-evenly

Arquivo(s) utilizado(s): alinhamento_grid.css

As colunas têm largura definida, e o espaço restante do container é distribuído horizontalmente entre a grade e suas extremidades.

CSS

```
.trilhas {
  grid-template-columns: repeat(2, 120px);
  grid-template-rows: repeat(2, 60px);
  justify-content: space-evenly;
}
```

30. align-content

align-content controla o alinhamento do Grid inteiro no eixo vertical. Assim como justify-content, atua sobre as trilhas como conjunto, e não sobre o conteúdo individual de cada célula.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.css

Como o container possui altura maior que a soma das linhas, existe espaço disponível para distribuir verticalmente.

CSS

```
.trilhas {
  min-height: 240px;
  grid-template-columns: repeat(2, 120px);
  grid-template-rows: repeat(2, 60px);
  align-content: space-evenly;
}
```

31. place-content

place-content combina justify-content e align-content. É útil quando queremos aplicar o mesmo tipo de alinhamento ao Grid nos dois eixos.

Relação com o exemplo fornecido

Arquivo(s) utilizado(s): alinhamento_grid.css

O arquivo enviado demonstra separadamente justify-content e align-content. A combinação equivalente poderia ser expressa com place-content quando os valores fossem iguais.

CSS

```
.trilhas {
  justify-content: space-evenly;
  align-content: space-evenly;
}
```

32. justify-self

justify-self permite controlar o alinhamento horizontal de um único item, sem alterar os demais.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.css

Os itens Início e Fim recebem alinhamentos individuais, enquanto o container define um comportamento geral.

CSS

```
.itens .inicio {
  justify-self: start;
}

.itens .fim {
  justify-self: end;
}
```

33. align-self

align-self controla o alinhamento vertical de um item específico.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.css

Os itens podem assumir comportamentos diferentes dentro da mesma grade: início, fim ou esticar.

CSS

```
.itens .inicio {  
    align-self: start;  
}  
  
.itens .fim {  
    align-self: end;  
}  
  
.itens .esticar {  
    align-self: stretch;  
}
```

34. place-self

place-self combina justify-self e align-self para um item específico.

Exemplo real

Arquivo(s) utilizado(s): alinhamento_grid.html / alinhamento_grid.css

Somente o item com classe destaque é centralizado nos dois eixos. O outro permanece com o comportamento padrão.

HTML

```
<div class="grade individual">  
    <div class="item">Padrão</div>  
    <div class="item destaque">place-self: center</div>  
</div>
```

CSS

```
.destaque {  
    place-self: center;  
}
```

35. minmax()

A função `minmax()` define um limite mínimo e um máximo para uma trilha. Ela é especialmente importante na criação de grades responsivas, pois permite impedir que uma coluna fique pequena demais enquanto ainda permite que ela cresça.

Exemplo real

Arquivo(s) utilizado(s): tamanho_trilhas.css

A terceira coluna terá pelo menos 140px e poderá crescer até ocupar uma fração do espaço disponível.

CSS

```
.conteudo {
  grid-template-columns: min-content max-content minmax(140px, 1fr);
}
```

36. auto-fit

`auto-fit` permite que o Grid ajuste automaticamente a quantidade de colunas que cabe no espaço disponível. Quando combinado com `minmax()`, cria uma grade que se adapta à largura do container.

Exemplo real de grade responsiva

Arquivo(s) utilizado(s): grade_responsiva.html / grade_responsiva.css

Em vez de criar uma regra para cada quantidade de colunas, o navegador calcula quantas cabem respeitando o limite mínimo.

HTML

```
<section class="cartoes" aria-label="Exemplos de cartões">
  <article class="cartao">
    <h2>Layout fluido</h2>
    <p><code>repeat(auto-fit, minmax(...))</code> cria quantas colunas couberem.</p>
  </article>
  ...
</section>
```

CSS

```
.cartoes {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(min(100%, 230px), 1fr));
  grid-auto-rows: minmax(170px, auto);
  gap: 18px;
}
```

37. auto-fill

`auto-fill` também participa da criação de grades responsivas. O documento-base destaca que `auto-fill` e `auto-fit` são semelhantes, mas podem apresentar diferenças quando existe espaço para trilhas vazias. Nos arquivos enviados, o exemplo implementado usa `auto-fit`.

Uso de auto-fit no material fornecido

Arquivo(s) utilizado(s): tamanho_trilhas.css

Como não foi fornecido um CSS que use `auto-fill`, o documento não substitui esse exemplo por um código inventado. O conceito é apresentado e o código real enviado é usado para demonstrar a solução efetivamente implementada.

CSS

```
.ajuste {  
  grid-template-columns:  
    repeat(auto-fit, minmax(min(100%, 150px), 1fr));  
}
```

38. Exemplo de Grid Responsivo

O arquivo `grade_responsiva.html` apresenta cartões que se adaptam ao espaço disponível. Além de `auto-fit` e `minmax()`, ele demonstra um cartão que ocupa duas colunas em telas maiores e volta ao fluxo normal em telas pequenas.

HTML + CSS reais

Arquivo(s) utilizado(s): `grade_responsiva.html` / `grade_responsiva.css`

Esse exemplo é uma referência prática para galerias, cards e componentes que precisam funcionar em diferentes larguras.

HTML

```
<!DOCTYPE html>  
<html lang="pt-br">  
  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Grid: grade responsiva</title>  
  <link rel="stylesheet" href="../../css/grid/grade_responsiva.css">  
</head>  
  
<body>  
  <main>  
    <h1>Grade responsiva de cartões</h1>  
    <p>As colunas se ajustam ao espaço disponível, sem precisar definir uma coluna por dispositivo.</p>  
  
    <section class="cartoes" aria-label="Exemplos de cartões">  
      <article class="cartao">  
        <h2>Layout fluido</h2>  
        <p><code>repeat(auto-fit, minmax(...))</code> cria quantas colunas couberem.</p>  
      </article>  
      <article class="cartao">  
        <h2>Largura mínima</h2>  
        <p>0 cartão mantém um tamanho legível e cresce quando há espaço disponível.</p>  
      </article>  
      <article class="cartao">  
        <h2>Uma coluna especial</h2>  
        <p>0 destaque pode ocupar mais trilhas com <code>grid-column: span 2</code>.</p>  
      </article>  
      <article class="cartao destaque">  
        <h2>Destaque</h2>  
        <p>Este cartão ocupa duas colunas em telas largas e volta ao fluxo normal em telas estreitas.</p>  
      </article>  
      <article class="cartao">  
        <h2>Sem largura fixa</h2>  
        <p>Use unidades flexíveis e limites mínimos para evitar overflow.</p>  
      </article>  
    </section>  
  </main>  
</body>
```

```
</html>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 24px;
  background: #eef3f7;
  color: #1c2c3b;
  font-family: Arial, sans-serif;
}

main {
  max-width: 1100px;
  margin: 0 auto;
}

.cartoes {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 230px), 1fr));
  grid-auto-rows: minmax(170px, auto);
  gap: 18px;
  margin-top: 28px;
}

.cartao {
  padding: 22px;
  background: white;
  border: 1px solid #d7e1e9;
  border-radius: 12px;
  box-shadow: 0 4px 14px #253c5010;
}

.cartao h2 {
  margin-top: 0;
  color: #2463a6;
}

.destaque {
  grid-column: span 2;
  background: #2463a6;
  color: white;
}

.destaque h2 {
  color: white;
}

@media (max-width: 560px) {
  .destaque {
    grid-column: auto;
  }
}
```

39. Grid e Media Queries

Embora muitos layouts responsivos possam ser construídos diretamente com Grid, Media Queries continuam sendo importantes quando a estrutura precisa mudar em determinados pontos de largura.

Exemplo real com dois breakpoints

Arquivo(s) utilizado(s): areas_template.css

O mesmo HTML é reorganizado em três situações: tela larga, tela intermediária e tela estreita. Isso demonstra que responsividade pode alterar a estrutura do Grid sem duplicar o conteúdo HTML.

CSS

```
@media (max-width: 720px) {  
  .pagina {  
    grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);  
    grid-template-areas:  
      "cabecalho cabecalho"  
      "navegacao navegacao"  
      "conteudo lateral"  
      "rodape rodape";  
  }  
}  
  
@media (max-width: 480px) {  
  .pagina {  
    grid-template-columns: 1fr;  
    grid-template-areas:  
      "cabecalho"  
      "navegacao"  
      "conteudo"  
      "lateral"  
      "rodape";  
  }  
}
```

40. Grid x Flexbox

É importante saber quando utilizar cada tecnologia. Flexbox é especialmente adequado para organizar elementos em uma dimensão, em uma linha ou coluna. Grid é especialmente adequado quando precisamos controlar linhas e colunas ao mesmo tempo.

Uma regra prática apresentada no documento-base é: Flexbox é excelente para uma dimensão; Grid é excelente para duas dimensões. Isso não significa que um substitua o outro: é comum utilizar ambos no mesmo projeto.

Exemplo Grid fornecido

Arquivo(s) utilizado(s): grid_basico.html / grid_basico.css

Nos arquivos enviados, não há um exemplo em Flexbox; por isso, a comparação permanece conceitual e o código demonstrado é o exemplo real de Grid.

HTML

```
<div class="grade colunas">  
  <div class="item">1fr</div>  
  <div class="item">2fr</div>  
  <div class="item">1fr</div>  
</div>
```

CSS

```
.colunas {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
```

41. Exemplo de página completa utilizando Grid

Em vez de reproduzir o exemplo genérico do documento-base, este material usa o arquivo `areas_template` como exemplo de página estruturada. Ele contém cabeçalho, navegação, conteúdo principal, conteúdo lateral e rodapé, todos posicionados por áreas nomeadas.

Página completa fornecida

Arquivo(s) utilizado(s): `areas_template.html` / `areas_template.css`

O ponto mais importante é perceber que o HTML permanece semanticamente organizado, enquanto o CSS define o desenho visual por meio de `grid-template-areas` e `grid-area`.

HTML

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Grid: áreas nomeadas</title>
  <link rel="stylesheet" href="../../css/grid/areas_template.css">
</head>

<body>
  <main>
    <h1>Áreas nomeadas</h1>
    <p>Com grid-template-areas, o desenho do layout fica visível no CSS.</p>

    <div class="pagina">
      <header class="cabecalho">Cabeçalho</header>
      <nav class="navigacao">Navegação</nav>
      <aside class="lateral">Conteúdo relacionado</aside>
      <article class="conteudo">
        <h2>Conteúdo principal</h2>
        <p>O mesmo HTML pode ser reorganizado alterando as áreas no CSS.</p>
      </article>
      <footer class="rodape">Rodapé</footer>
    </div>
  </main>
</body>

</html>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 24px;
  background: #f4f6f8;
  color: #1c2733;
  font-family: Arial, sans-serif;
```

```

}

main {
  max-width: 1000px;
  margin: 0 auto;
}

.pagina {
  display: grid;
  grid-template-columns: 220px minmax(0, 1fr) 200px;
  grid-template-areas:
    "cabecalho cabecalho cabecalho"
    "navegacao conteudo lateral"
    "rodape rodape rodape";
  gap: 14px;
  margin-top: 24px;
}

.pagina > * {
  padding: 24px;
  border-radius: 10px;
}

.cabecalho {
  grid-area: cabecalho;
  background: #245b8f;
  color: white;
}

.navegacao {
  grid-area: navegacao;
  background: #d8e6f2;
}

.lateral {
  grid-area: lateral;
  background: #f3dfbf;
}

.conteudo {
  grid-area: conteudo;
  min-height: 240px;
  background: white;
}

.rodape {
  grid-area: rodape;
  background: #273b4f;
  color: white;
}

@media (max-width: 720px) {
  .pagina {
    grid-template-columns: minmax(0, 1fr) minmax(0, 1fr);
    grid-template-areas:
      "cabecalho cabecalho"
      "navegacao navegacao"
      "conteudo lateral"
      "rodape rodape";
  }
}

```

```

@media (max-width: 480px) {
  .pagina {
    grid-template-columns: 1fr;
    grid-template-areas:
      "cabecalho"
      "navegacao"
      "conteudo"
      "lateral"
      "rodape";
  }
}

```

42. Principais propriedades do CSS Grid

A tabela abaixo reúne as propriedades apresentadas no documento-base e mostra como elas aparecem nos arquivos fornecidos.

Propriedade	Função	Exemplo fornecido
display: grid	Ativa o Grid	.grade { display: grid; }
grid-template-columns	Define as colunas	1fr 2fr 1fr
grid-template-rows	Define as linhas	70px 100px 130px
grid-template-areas	Define áreas nomeadas	"cabecalho cabecalho cabecalho"
grid-column	Posiciona/expand horizontalmente	grid-column: 1 / 4
grid-row	Posiciona/expand verticalmente	grid-row: 2 / 4
grid-area	Define uma área ou combina linhas	grid-area: 3 / 3 / 4 / 5
grid-auto-flow	Controla o fluxo automático	row / column / row dense
grid-auto-rows	Dimensiona linhas implícitas	minmax(65px, auto)
grid-auto-columns	Dimensiona colunas implícitas	minmax(100px, 1fr)
gap	Define espaçamento	gap: 10px
justify-items / align-items	Alinha itens dentro das células	center
justify-content / align-content	Alinha o Grid dentro do container	space-evenly
place-items / place-self	Atalhos de alinhamento	center

43. Principais funções e valores

Função/valor	Função	Exemplo real
repeat()	Repete trilhas	repeat(4, minmax(100px, 1fr))
minmax()	Define mínimo e máximo	minmax(140px, 1fr)
auto-fit	Ajusta a quantidade de colunas	repeat(auto-fit, minmax(...))
auto-fill	Preenche o espaço considerando trilhas possíveis	Conceito do documento-base; não há CSS fornecido usando-o
span	Faz um item ocupar várias trilhas	grid-column: span 2
fr	Divide o espaço disponível em frações	1fr 2fr 1fr
min-content	Usa o limite mínimo relacionado ao conteúdo	grid-template-columns: min-content ...
max-content	Dimensiona conforme o conteúdo	grid-template-columns: ... max-content ...
fit-content()	Controla o ajuste do tamanho ao conteúdo	Apresentado no HTML de tamanho_trilhas; sem uso correspondente no CSS fornecido

44. Atividade prática 1 — Analise a grade de dimensionamento

Use o arquivo tamanho_trilhas.html/tamanho_trilhas.css como base. Identifique no código quais colunas usam fr, quais usam min-content, max-content e minmax(), e explique o efeito de cada uma.

Trecho para análise

Arquivo(s) utilizado(s): tamanho_trilhas.css

Objetivo: relacionar o código CSS ao comportamento visual observado no navegador.

CSS

```
.fracoes {
  grid-template-columns: 1fr 2fr 1fr;
}

.conteudo {
  grid-template-columns: min-content max-content minmax(140px, 1fr);
}

.ajuste {
  grid-template-columns:
    repeat(auto-fit, minmax(min(100%, 150px), 1fr));
}
```

45. Atividade prática 2 — Analise a grade responsiva

Use grade_responsiva.html como base. Redimensione a janela do navegador e observe quando o cartão destaque deixa de ocupar duas colunas. Explique o papel de auto-fit, minmax() e da Media Query.

Trecho para análise

Arquivo(s) utilizado(s): grade_responsiva.css

Objetivo: perceber a diferença entre uma adaptação automática da grade e uma mudança explícita feita por Media Query.

CSS

```
.cartoes {
  display: grid;
}
```

```

    grid-template-columns:
      repeat(auto-fit, minmax(min(100%, 230px), 1fr));
  }

  .destaque {
    grid-column: span 2;
  }

  @media (max-width: 560px) {
    .destaque {
      grid-column: auto;
    }
  }
}

```

46. Atividade prática 3 — Reorganize a página por áreas

Use `areas_template.html` como base. Modifique somente `grid-template-areas` para criar uma organização diferente em uma largura de tela e, depois, adapte as áreas para telas menores. O objetivo é observar que o HTML pode permanecer igual enquanto o desenho da página muda no CSS.

Trecho para alteração

Arquivo(s) utilizado(s): `areas_template.css`

Objetivo: compreender a separação entre estrutura HTML e posicionamento visual feito pelo Grid.

CSS

```

.pagina {
  display: grid;
  grid-template-columns: 220px minmax(0, 1fr) 200px;
  grid-template-areas:
    "cabecalho cabecalho cabecalho"
    "navegacao conteudo lateral"
    "rodape rodape rodape";
}

```

47. Resumo

O CSS Grid é um sistema de layout baseado em linhas e colunas. A propriedade principal é `display: grid`. A partir dela, podemos definir trilhas, espaçamentos, posicionamento, áreas, alinhamento e comportamento responsivo.

As propriedades fundamentais para começar são:

- `display: grid`
- `grid-template-columns`
- `grid-template-rows`
- `gap`

Depois podemos avançar para:

- `grid-column`
- `grid-row`
- `grid-area`
- `grid-template-areas`
- `grid-auto-flow`
- `grid-auto-rows`
- `grid-auto-columns`

Para criar layouts flexíveis e responsivos, os exemplos fornecidos demonstram:

- `fr`

- `repeat()`
- `minmax()`
- `auto-fit`
- Media Queries
- linhas e colunas nomeadas

Padrão importante: Nos exemplos enviados, um dos padrões centrais para grades responsivas é: `repeat(auto-fit, minmax(min(100%, 230px), 1fr))` combinado com `gap`. Esse padrão permite que a quantidade de colunas se adapte ao espaço disponível sem precisar definir manualmente uma regra para cada dispositivo.

Conceito final: CSS Grid permite organizar elementos em linhas e colunas, controlar suas dimensões, posicioná-los em diferentes áreas e criar layouts que se adaptam ao espaço disponível. Quando combinado com Flexbox, Media Queries e unidades relativas, torna-se uma ferramenta central para a construção de interfaces Web responsivas.

Arquivos utilizados como referência prática

Arquivo	Conteúdo aproveitado
grid_basico.html / grid_basico.css	linhas, colunas, fr, repeat(), minmax()
tamanho_trilhas.html / tamanho_trilhas.css	dimensionamento, min-content, max-content, minmax(), auto-fit
posicionamento_itens.html / posicionamento_itens.css	grid-column, grid-row, span, grid-area e linhas nomeadas
areas_template.html / areas_template.css	grid-template-areas, grid-area e responsividade
fluxo_automatico.html / fluxo_automatico.css	grid-auto-flow, grid-auto-rows, grid-auto-columns e trilhas implícitas
alinhamento_grid.html / alinhamento_grid.css	justify-items, align-items, place-items, justify-content, align-content e place-self
grade_responsiva.html / grade_responsiva.css	auto-fit, minmax(), grid-column: span e Media Query
guia_completo_grid.html	roteiro integrador dos conceitos; o CSS correspondente não foi fornecido nesta etapa